



GUÍA DE LABORATORIO N° 02

Modelos de Clasificación

1. Objetivos

Objetivo general:

Implementar, evaluar y comparar modelos de clasificación supervisada mediante el dataset Breast Cancer Wisconsin, aplicando métricas de evaluación estándar, validación cruzada estratificada y técnicas de selección de modelos.

Objetivos específicos:

- Construir pipelines de clasificación con Regresión Logística, Árbol de Decisión, SVM y K-Nearest Neighbors (KNN).
- Evaluar el rendimiento con matriz de confusión, Accuracy, Precision, Recall, F1-Score y curva ROC/AUC.
- Aplicar Stratified K-Fold Cross-Validation (5-fold) para obtener estimaciones robustas de rendimiento.
- Comparar modelos utilizando una matriz de selección basada en múltiples criterios.
- Diagnosticar y manejar desbalance de clases con `class_weight="balanced"`.
- Interpretar coeficientes, importancia de features y fronteras de decisión según cada modelo.

2. Desarrollo del laboratorio

2.1 Dataset: Breast Cancer Wisconsin

El dataset Breast Cancer Wisconsin contiene 569 muestras de biopsias con 30 features numéricas computadas a partir de imágenes digitalizadas de aspiración con aguja fina (FNA). El target es binario: maligno (0) o benigno (1).

Feature	Descripción
mean radius	Radio medio de las células
mean texture	Desviación estándar de valores de escala de grises
mean perimeter	Perímetro medio
mean area	Área media
mean smoothness	Variación local de longitudes de radio
mean compactness	perímetro ² /área - 1.0
mean concavity	Severidad de porciones cóncavas del contorno
... (30 features)	10 features × 3 estadísticos (mean, error, worst)
target	0 = maligno (212), 1 = benigno (357)

Nota: Las clases están moderadamente desbalanceadas (62.7% benigno, 37.3% maligno). En diagnóstico médico, minimizar falsos negativos (FN) es crítico: un tumor maligno clasificado como benigno puede ser fatal.

2.2 Preparación de datos

Se realiza una partición estratificada 80/20 con `train_test_split` (`stratify=y`, `random_state=42`) para mantener la proporción de clases en ambos conjuntos. Se aplica `StandardScaler` dentro de cada pipeline para evitar fuga de datos (data leakage).

Funciones auxiliares:

- **eval_classifier():** Genera predicciones, calcula métricas (Accuracy, Precision, Recall, F1), muestra la matriz de confusión con heatmap y el `classification_report` completo.
- **plot_roc():** Calcula y grafica la curva ROC con el AUC para modelos con `predict_proba` o `decision_function`.

2.3 Modelo 1: Regresión Logística

Modelo lineal que estima la probabilidad de pertenencia a una clase mediante la función sigmoide $\sigma(z) = 1/(1+e^{-z})$. Para clasificación binaria, si $\sigma(\theta^T x) \geq 0.5$ se predice clase 1, de lo contrario clase 0. La optimización minimiza la función Log-Loss (entropía cruzada binaria) con regularización L2 por defecto (parámetro $C=1.0$).

Actividades:

- Entrenar un Pipeline(StandardScaler → LogisticRegression) con max_iter=1000.
- Evaluar con Stratified 5-Fold Cross-Validation (scoring: accuracy, f1).
- Interpretar los coeficientes estandarizados: identificar las features con mayor peso para maligno y benigno.
- Generar la curva ROC y calcular el AUC.
- Analizar la matriz de confusión: ¿cuántos falsos negativos (maligno clasificado como benigno)?

Conceptos clave:

- **Frontera de decisión:** Regresión Logística produce una frontera lineal en el espacio de features.
- **Regularización C:** Valores menores de C producen mayor regularización (menor varianza, mayor sesgo).

2.4 Modelo 2: Árbol de Decisión

El algoritmo CART (Classification and Regression Trees) construye particiones recursivas del espacio de features mediante divisiones binarias. En cada nodo selecciona la feature y el umbral que maximizan la reducción de impureza, medida por el índice Gini: $G(t) = 1 - \sum p_k^2$, o por la Entropía: $H(t) = -\sum p_k \log_2(p_k)$.

Actividades:

- Entrenar un DecisionTreeClassifier con criterion="gini" y sin restricciones iniciales.
- Evaluar con Stratified 5-Fold CV y métricas completas.
- Aplicar pre-poda: ajustar max_depth=5, min_samples_split=10, min_samples_leaf=5 y comparar rendimiento.
- Aplicar post-poda: utilizar ccp_alpha (cost-complexity pruning) con búsqueda del alpha óptimo.
- Analizar feature_importances_: identificar las features más discriminativas.

Conceptos clave:

- **Pre-poda:** Restricciones aplicadas durante la construcción del árbol (`max_depth`, `min_samples_split`, `min_samples_leaf`).
- **Post-poda (`ccp_alpha`):** Recorta ramas después de construir el árbol completo; el parámetro α controla el balance entre complejidad y ajuste.
- **Sobreajuste:** Un árbol sin restricciones memoriza el entrenamiento; la poda mejora la generalización.

2.5 Modelo 3: Support Vector Machine (SVM)

SVM busca el hiperplano de máximo margen $\gamma = 2/\|w\|$ que separa las clases. Con margen suave (soft margin), permite violaciones controladas por variables de holgura ξ_i : minimiza $\frac{1}{2}\|w\|^2 + C \cdot \sum \xi_i$. El parámetro C controla el trade-off entre margen y errores de clasificación.

Actividades:

- Entrenar un Pipeline (`StandardScaler` → `SVC(kernel="rbf", C=1.0, probability=True)`).
- Evaluar con Stratified 5-Fold CV y métricas completas.
- Comparar kernels: lineal vs RBF vs polinomial — evaluar cuál captura mejor la estructura de los datos.
- Analizar el efecto de C : valores $\{0.01, 0.1, 1, 10, 100\}$ con kernel RBF.
- Analizar el efecto de γ : valores $\{0.001, 0.01, 0.1, 1\}$ con kernel RBF.

Conceptos clave:

- **Kernel trick:** Transforma el espacio de features a una dimensión superior donde las clases son linealmente separables, sin calcular explícitamente la transformación.
- **RBF (Radial Basis Function):** $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$. γ alto → radio de influencia pequeño → más sobreajuste.
- **Estandarización:** Crítica para SVM ya que los márgenes dependen de las distancias en el espacio de features.

2.6 Modelo 4: K-Nearest Neighbors (KNN)

KNN es un algoritmo de aprendizaje basado en instancias (lazy learning). No construye un modelo explícito; clasifica una nueva instancia por voto mayoritario de sus K vecinos más cercanos según una métrica de distancia (Euclidiana, Manhattan, Minkowski).

Actividades:

- Entrenar un Pipeline(StandardScaler → KNeighborsClassifier(n_neighbors=5)).
- Evaluar con Stratified 5-Fold CV y métricas completas.
- Búsqueda del K óptimo: evaluar $K \in \{1, 3, 5, 7, 9, 11, 15, 21\}$ y graficar accuracy vs K.
- Comparar métricas de distancia: Euclidiana vs Manhattan vs Minkowski ($p=3$).
- Analizar la maldición de la dimensionalidad: aplicar PCA para reducir a 2 componentes y comparar rendimiento.

Conceptos clave:

- **Regla empírica para K:** $K \approx \sqrt{n}$ (para $n=455$ entrenamiento, $K \approx 21$). Se debe usar K impar para clasificación binaria.
- **Maldición de la dimensionalidad:** Con 30 features, las distancias tienden a ser similares; la estandarización y reducción de dimensionalidad son importantes.
- **Pesos por distancia:** weights="distance" pondera la contribución de cada vecino inversamente a su distancia.

2.7 Métricas de evaluación

Todas las métricas se derivan de la matriz de confusión (TP, FP, FN, TN):

Métrica	Fórmula	Interpretación en contexto médico
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Proporción total de aciertos
Precision	$TP/(TP+FP)$	De los diagnosticados maligno, ¿cuántos lo son realmente?
Recall (Sensibilidad)	$TP/(TP+FN)$	De los realmente malignos, ¿cuántos se detectaron?
F1-Score	$2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$	Media armónica — balancea Precision y Recall
AUC	Área bajo curva ROC	1.0 = separación perfecta; 0.5 = aleatorio

En diagnóstico de cáncer, Recall es la métrica más crítica: un falso negativo (maligno no detectado) tiene consecuencias más graves que un falso positivo (benigno diagnosticado como maligno, que solo genera pruebas adicionales).

2.8 Comparación final de modelos

Se comparan los 4 modelos mediante:

- Tabla resumen con Accuracy, Precision, Recall, F1-Score y AUC.
- Gráfico de barras comparativo de las 5 métricas.
- Curvas ROC superpuestas en un solo gráfico.
- Matriz de selección con criterios: precisión, velocidad, interpretabilidad, escalabilidad, robustez a outliers.

Modelo	Fortalezas	Limitaciones	Caso de uso
Reg. Logística	Interpretable, probabilístico, rápido	Asume linealidad en log-odds	Clasificación binaria con interpretabilidad
Árbol de Decisión	Interpretable, no requiere escalar	Propenso a sobreajuste	Reglas de negocio explicables
SVM	Efectivo en alta dimensión, robusto	Lento en datasets grandes, menos interpretable	Datasets de dimensión media-alta
KNN	Simple, no paramétrico	Lento en predicción, sensible a dimensionalidad	Datasets pequeños con features normalizadas

3. Ejercicios propuestos

Ejercicio 1 — Optimización de hiperparámetros con GridSearchCV

Para cada uno de los 4 modelos, realice una búsqueda exhaustiva de hiperparámetros con GridSearchCV (cv=5, scoring="f1").

Parámetros sugeridos:

- Logistic Regression(C=[0.01,0.1,1,10,100])
- DecisionTree (max_depth=[3,5,7,10,None], min_samples_leaf=[1,5,10]),
- SVM (C=[0.1,1,10], gamma=[0.001,0.01,0.1]),
- KNN (n_neighbors=[3,5,9,15,21], weights=["uniform","distance"]).

Compare los mejores modelos optimizados.

Ejercicio 2 — Manejo de desbalance de clases

Entrene los 4 modelos con `class_weight="balanced"` (o su equivalente). Compare las métricas (especialmente Recall de la clase maligna) con los modelos sin balanceo. Además, aplique SMOTE (`from imblearn.over_sampling import SMOTE`) para sobremuestrear la clase minoritaria y evalúe si mejora el rendimiento.

Ejercicio 3 — Dataset de clasificación propio

Seleccione un dataset de clasificación de Kaggle, UCI u otra fuente (mínimo 500 registros, 5 features). Puede ser binario o multiclase. Aplique los 4 modelos de esta sesión + al menos uno adicional (Random Forest, Gradient Boosting o Naive Bayes). Documente: preprocesamiento, métricas, curvas ROC, matriz de selección y justificación del mejor modelo.